

Principles and properties	Structures
✓ Versatility	✓ Module
Conceptual integrity	Dependency
✓ Independently changeable	Process
Automatic propagation	Data access
Buildability	
✓ Growth accommodation	
✓ Entropy resistance	

CHAPTER ELEVEN

GNU Emacs: Creeping Featurism Is a Strength

Jim Blandy

I use Emacs, which might be thought of as a thermonuclear word processor. It was created by Richard Stallman; enough said. It is written in Lisp, which is the only computer language that is beautiful. It is colossal, and yet it only edits straight ASCII text files, which is to say, no fonts, no boldface, no underlining.... If you are a professional writer—i.e., if someone else is getting paid to worry about how your words are formatted and printed—Emacs outshines all other editing software in approximately the same way that the noonday sun does the stars. It is not just bigger and brighter; it simply makes everything else vanish.

—Neal Stephenson

THE GNU EMACS TEXT EDITOR IS UNMATCHED IN ITS NOTORIETY. Its proponents swear nothing else comes close, and are oddly resistant to the charms of more modern alternatives. Its detractors call it obscure, complex, and outdated compared to more widely used development environments, such as Microsoft's Visual Studio. Even its fans blame their wrist injuries on its contorted keyboard command set.

Emacs provokes such strong reactions partly because there's so much of Emacs to react to. The current Emacs sources include 1.1 million lines of code written in Emacs's own programming language, Emacs Lisp. This corpus includes code to help you edit programs in C, Python, and other languages, as you might expect from a programmer's text editor. But it also includes code to help you debug running programs, collaborate with other programmers, read electronic mail and news, browse and search directories, and solve symbolic algebra problems.

Looking under the hood, the story gets stranger. Emacs Lisp has no object system, its module system is just a naming convention, all the fundamental text editing operations use implicit global arguments, and even local variables aren't quite local. Almost every software engineering principle that has become generally accepted as useful and valuable, Emacs flouts. The code is 24 years old, huge, and written by hundreds of different people. By rights, the whole thing should blow up.

But it works—and works rather well. Its bazaar of features grows; the user interface acquires addictive new behavior; and the project survives broad changes to its fundamental architecture, infrequent releases, leadership conflicts, and forks. What is the source of this vigor? What are its limitations?

And finally, what can other software learn from Emacs? When we encounter a new architecture that shares some goal with Emacs, what question can we ask to gauge its success? Over the course of this chapter, I'll pose three questions that I think capture the most valuable characteristics of Emacs's architecture.

Emacs in Use

Before discussing its architecture, let's look briefly at what Emacs is. It's a text editor that you can use much like any other. When you invoke Emacs on a file, a window appears, displaying the file's contents. You can make your changes, save the revised contents, and exit. However, Emacs is not very effective when used this way: it is slower to start than other popular text editors, and its strengths don't come to bear. When I need to work in this fashion, I don't use Emacs.

Emacs is meant to be started once, when you begin work, and then left running. You can edit as many files as you like within one Emacs session, saving changes as you go. Emacs can keep files in memory without displaying them, so what you see reflects what you're working on at present, but your other tasks are ready to be called up just as you left them. The experienced Emacs user closes files only if the computer seems to be running low on memory, so a long-running Emacs session may have hundreds of files open. The screenshot in Figure 11-1 shows an Emacs session with two frames open. The left frame is split into three windows, showing the Emacs splash screen, a browsable directory listing, and a Lisp interaction buffer. The right frame has a single window, displaying a buffer of source code.

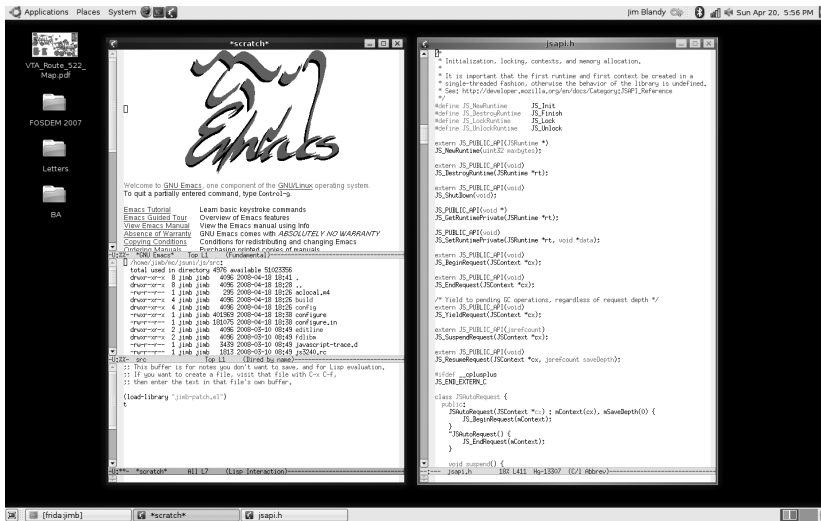


FIGURE 11-1. Emacs in use

There are three essential kinds of objects involved here: frames, windows, and buffers.

Frames are what Emacs calls the windows of your computer’s graphical user interface. The screenshot shows two frames side by side. If you use Emacs in a text terminal, perhaps via a *telnet* or *ssh* connection, that terminal is also an Emacs frame. Emacs can manage any number of graphical frames and terminal frames simultaneously.

Windows are subdivisions of frames.* New windows are created only by dividing existing windows in two, and deleting a window returns its space to the adjacent windows; as a consequence, a frame’s windows (or window) always fill its space completely. There is always a currently selected window, to which keyboard commands apply. Windows are lightweight; in typical use, they tend to come and go frequently.

Finally, *buffers* hold editable text. Emacs holds each open file’s text in a buffer, but a buffer need not be associated with a file: it might contain the results of a search, online documentation, or simply text entered by hand and not yet saved to any file. Each window displays the contents of some buffer, and a buffer may appear in zero, one, or more windows.

It’s important to understand that, aside from the mode line at the bottom of each window and other similar decorations, the only way Emacs ever displays text to users is by placing it in a buffer and then displaying that buffer in some window. Help messages, search results, directory

* Note that what most graphical user interfaces call a window, Emacs calls a frame, since Emacs uses the term “window” as described earlier. This is unfortunate, but Emacs’s terminology was established well before the widespread use of graphical user interfaces, and Emacs’s maintainers seem uninclined to change it.

listings, and the like all go into buffers with appropriately chosen names. This may seem like a cheap implementation trick—it does simplify Emacs internally—but it’s actually quite valuable because it means that these different kinds of content are all ordinary editable text: you can use the same commands to navigate, search, organize, trim, and sort this data that are available to you in any other text buffer. Any command’s output can serve as any another command’s input. This is in contrast with environments such as Microsoft Visual Studio, where the results of, say, a search can only be used in the ways the implementors anticipated would be useful. But Visual Studio is not alone in this; most programs with graphical user interfaces have the same shortcoming.

For example, in the screenshot, the middle window of the frame on the left shows a directory listing. Like most directory browsers, this window provides terse keyboard commands for copying, deleting, renaming, and comparing files, selecting groups of files with globbing patterns or regular expressions, and (of course) visiting the files in Emacs. But unlike most directory browsers, the listing itself is plain text, held in a buffer. All the usual Emacs search facilities (including the excellent *incremental search* commands) apply. I can readily cut and paste the listing into a temporary buffer, delete the metadata on the left to get a plain list of names, use a regular expression search to winnow out the files I’m not interested in, and end up with a list of filenames to pass to some new operation. Once you’ve gotten used to working this way, using ordinary directory browsers becomes annoying: the information displayed feels out of reach, and the commands feel constrained. In some cases, even composing commands in the shell can feel like working in the dark because it’s not as easy to see the intermediate results of your commands as you go.

This suggests the first question of the three I promised at the beginning of this chapter, a question we can ask of any user interface we encounter: *how easy is it to use the results of one command as input to another?* Do the interface’s commands compose with each other? Or have the results reached a dead end once they’ve been displayed? I would argue that one of the reasons many programmers have been so loyal to the Unix shell environment, despite its gratuitous inconsistencies, anorexic data model, and other weaknesses, is that nearly everything can be coaxed into being part of some larger script. It’s almost easier to write a program that is readily scriptable than not, so dead ends are rare. Emacs achieves this same goal, although it takes a radically different route.

Emacs’s Architecture

Emacs’s architecture follows the widely used *Model-View-Controller* pattern for interactive applications, as shown in Figure 11-2. In this pattern, the *Model* is the underlying representation of the data being manipulated; the *View* presents that data to the user; and the *Controller* takes the user’s interactions with the View (keystrokes, mouse gestures, menu selections, and so on) and manipulates the Model accordingly. Throughout this chapter, I’ll capitalize Model, View, and Controller when I’m referring to elements of this pattern. In Emacs,

the Controller is almost entirely Emacs Lisp code. Lisp primitives manipulate buffer contents (the Model) and the window layout. Redisplay code (the View) updates the display without explicit guidance from the Lisp code. Neither the buffer’s implementation nor the redisplay code can be customized by Lisp code.

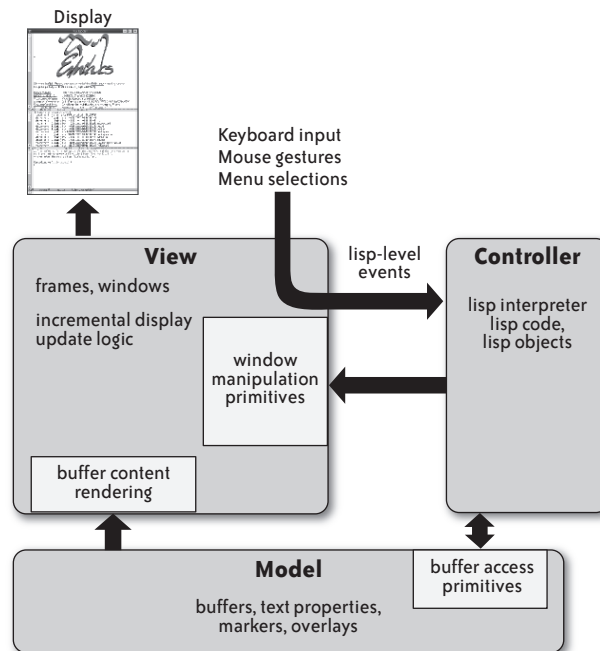


FIGURE 11-2. The Model-View-Controller pattern in Emacs

The Model: Buffers

Emacs edits text files, so the heart of Emacs’s Model is the *buffer* type, which holds text. A buffer is simply a flat string, where newline characters mark line endings; it is not a list of lines, nor is it a tree of nodes, like the document object model that web browsers use to represent HTML documents. Emacs Lisp has primitive operations on buffers that insert and delete text, extract portions of buffer text as strings, and search for matches of exact strings or regular expressions. A buffer can hold characters from a wide variety of character sets, including those needed to write most Asian and European scripts.

Each buffer has a *mode*, which specializes the buffer’s behavior for editing a given kind of text. Emacs includes modes for editing C code, XML text, and hundreds of other kinds of content. At the Lisp level, modes use *buffer-local key bindings* to make mode-specific commands available to the user, and use *buffer-local variables* to maintain state specific to that buffer.

Used together, these features make a buffer's mode closely analogous to an object's class: the mode determines which commands are available and provides the variables upon which those commands' Lisp implementations depend.

For each buffer, the text-editing primitives maintain an "undo" log, which holds enough information to revert their effects. The undo log remembers the boundaries between user commands, so a user command that carries out many primitive operations can be undone as a unit.

When Emacs Lisp code operates on a buffer, it can use integers to specify character positions within a buffer's text. This is a simple approach, but insertions and deletions before a given piece of text cause its numeric position to change from one moment to the next. To track positions within a buffer as its contents are modified, Lisp code can create *marker* objects, which float with the text they're next to. Any primitive operation that accepts integers as buffer positions also accepts markers.

Lisp code can attach *text properties* to the characters in a buffer. A text property has a name and a value, and both can be arbitrary Lisp objects. Logically, each character's properties are independent, but Emacs's representation of text properties stores a run of consecutive characters with identical text properties efficiently, and Emacs Lisp includes primitives to quickly find positions where text properties change, so in effect, text properties mark ranges of text. Text properties can specify how Emacs should display the text, and also how Emacs should respond to mouse gestures applied to it. Text properties can even specify special keyboard commands available to the user only when the cursor is on that text. A buffer's undo log records changes to its text properties, in addition to the changes to the text itself. Emacs Lisp strings can have text properties, too: extracting text from a buffer as a string and inserting that string into a buffer elsewhere carries the text properties along.

An *overlay* represents a contiguous stretch of text in a buffer. The start and end of an overlay float with the text around them, as markers would. Like text properties, overlays can affect the display and mouse sensitivity of the text they cover, but unlike text properties, overlays are not considered part of the text: strings cannot have overlays, and the undo log doesn't record changes to overlay endpoints.

The View: Emacs's Redisplay Engine

As the user edits text and manipulates windows, Emacs's redisplay engine takes care of keeping the display up to date. Emacs redisplay has two important characteristics:

Emacs updates the display automatically

Lisp code need not specify how to update the display. Rather, it is free to manipulate buffer text, properties, overlays, and window configurations as needed, without regard for which portions of the display will become out of date. When the time comes, Emacs's redisplay code looks over the accumulated changes and finds an efficient set of drawing operations that will bring the display into line with the new state of the Model. By relieving Lisp code

of responsibility for managing the display, Emacs greatly simplifies the task of writing correct extensions.

Emacs updates the display only when waiting for user input

A single command may apply any number of buffer manipulation primitives, affecting arbitrary portions of the buffer—or perhaps of many different buffers. Similarly, the user may invoke a keyboard macro that carries out a long series of prerecorded commands. In cases like these, rather than showing the intermediate states of the buffer flickering by as it works, Emacs puts off updating the display until it needs to pause for user input, and refreshes the display in a single step at that point. This relieves Lisp programmers from the temptation to optimize or arrange their uses of buffer and window primitives to get smoother display behavior. The simplest code will display well.

By updating the display to reflect the effects of editing primitives automatically and efficiently, Emacs drastically simplifies the work of Lisp authors. Some other systems have picked up on this behavior: most notably, JavaScript programs embedded in web pages can edit the page's contents, and the browser will update the display accordingly. However, a surprising number of systems that invite extensions require careful cooperation between the extension code and the display update code, laying this burden on the extension author's back.

When Emacs was first written, its audience used computers via standalone terminals connected to the computer over serial lines or modems. These connections often were not terribly fast, so a text editor's ability to find the optimal series of drawing operations sufficient to update the screen could have a big impact on the editor's usability. Emacs went to a great deal of trouble to do well here, having a hierarchy of update strategies ranging from the quick-but-limited to the more-work-but-exhaustive. The latter used the same dynamic programming techniques employed by file comparison programs such as *diff* to find the smallest set of operations sufficient to transform the old screen into the new screen. In modern times, although Emacs still uses these algorithms to minimize update work, most of this effort is wasted, as faster processors and faster links between the computer and the display allow simpler algorithms to perform perfectly well.

The Controller: Emacs Lisp

The heart of Emacs is its implementation of its own dialect of Lisp. In Emacs's realization of the Model-View-Controller pattern, Lisp code dominates the Controller: almost every command you invoke, whether from the keyboard, a menu, or by name, is a Lisp function. Emacs Lisp is the key to Emacs's ability to successfully accommodate the wide range of functionality that Emacs has grown to offer.

THE FIVE-MINUTE LISP TUTORIAL

People new to Lisp often find the language hard to read. This is mostly because Lisp has fewer syntactic constructs than most languages, but works those constructs harder to get an even richer vocabulary of features. To give you a reader's intuition, consider the following rules for converting code from Python to Lisp:

1. Write control constructs as if they were function calls. That is, `while x*y < z: q()` becomes `while (x*y < z, q())`. This is just a change in notation; it's still a while loop.
2. Write uses of infix operators as if they were also function calls, where the functions have odd-looking names. The expression `x*y < z` would become `<(* (x, y), z)`. Now any parentheses present only for grouping (as opposed to those that surround function arguments) are redundant; remove them.
3. Now everything looks like a function call, including control constructs and primitive operations. Move the opening parenthesis of every "call" from after the function name to before the function name, and drop the commas. For example, `f(x, y, z)` becomes `(f x y z)`. To continue the previous example, `<(* (x, y), z)` becomes `(< (* x y) z)`.

Taking all three rules together, the Python code `while x*y < z: q()` becomes the Lisp code `(while (< (* x y) z) (q))`. This is a proper Emacs Lisp expression, with the same meaning as the original Python statement.

There's more, obviously, but this is the essence of Lisp's syntax. The bulk of Lisp is simply a vocabulary of functions (such as `<` and `*`) and control structures (such as `while`), all used as shown here.

Here is the definition of an interactive Emacs command to count words in the currently selected region of text. If I explain that `"\<"` is an Emacs regular expression matching the beginning of a word, you can probably read through it:

```
(defun count-region-words (start end)
  "Count the words in the selected region of text."
  (interactive "r")
  (save-excursion
    (let ((count 0))
      (goto-char start)
      (while (re-search-forward "\<" end t)
        (setq count (+ count 1))
        (forward-char 1))
      (message "Region has %d words." count))))
```

There's an argument to be made that Lisp gives its human readers too few visual cues as to what's really going on in the code, and that it should abandon this bland syntax in favor of one that makes more apparent distinctions between primitive operations, function calls, control structures, and the like. However, some of Lisp's most important features (which I can't go into here) depend

fundamentally on this syntactic uniformity; the many attempts that have been made to leave it behind generally haven't fared well. With experience, many Lisp programmers come to feel that the syntax is a reasonable price to pay for the language's power and flexibility.

In Emacs, a command is simply an Emacs Lisp function that the programmer has marked (with a brief annotation at the top of the definition) as suitable for interactive use. The name of the command that the user types after `[Meta]+[x]` is the name of the Lisp function. *Keymaps* bind key sequences, mouse clicks, and menu selections to commands. Core Emacs code takes care of looking up the user's keys and mouse gestures in the appropriate keymaps and dispatching control to the indicated command. Along with the automatic display management provided by Emacs's View, the keymap dispatching process means that Lisp code is almost never responsible for handling events from an event loop, a welcome relief for many user interface programmers.

Emacs and its Lisp have some critical characteristics:

- Emacs Lisp is light on bureaucracy. Small customizations and extensions are easy: one-line expressions placed in the `.emacs` file in your home directory, which Emacs loads automatically when it starts, can load existing packages of Lisp code, set variables that affect Emacs's behavior, redefine key sequences, and so on. You can write a useful command in under a dozen lines, including its online documentation. (See the "The Five-Minute Lisp Tutorial" sidebar for the complete definition of a useful Emacs command.)
- Emacs Lisp is interactive. You can enter definitions and expressions into an Emacs buffer and evaluate them immediately. Revising a definition and reevaluating it puts your changes in place; there is no need to recompile or restart Emacs. Emacs is, in effect, an integrated development environment for Emacs Lisp programs.
- The Emacs Lisp programs you write yourself are first-class citizens. Every nontrivial editing feature of Emacs itself is written in Lisp, so all the primitive functions and libraries Emacs itself needs are equally available for use in your own Lisp code. Buffers, windows, and other editing-related objects appear as ordinary values in Lisp code. You can pass them to and return them from functions, store them in data structures, and so on. Although Emacs's Model and View components are hardcoded, Emacs arrogates no special privileges to itself within the Controller.
- Emacs Lisp is a full programming language. It is comfortable to write reasonably large programs (hundreds of thousands of lines) in Emacs Lisp.
- Emacs Lisp is safe. Buggy Lisp code may cause an error to be reported, but it can't crash Emacs. The damage a bug can do is contained in other ways as well: for example, buffers' built-in undo logging lets you revert many unintended effects. This makes Lisp development more pleasant and encourages experimentation.

- Emacs Lisp code is easy to document. A function’s definition can include a *docstring* (for “documentation string”), text explaining the function’s purpose and use. Almost every function provided with Emacs has a docstring, meaning that help on a given facility is never more than a command away. And when Emacs displays a function’s docstring, it includes a hyperlink to the function’s source code, making Lisp code easy to browse as well. (Naturally, docstrings aren’t adequate for large Lisp packages, so those usually include a more traditionally structured manual as well.)
- Emacs Lisp has no module system. Instead, established naming conventions help unrelated packages avoid interfering with each other when loaded into the same Emacs session, so users can share packages of Emacs Lisp code with each other without the coordination or approval of Emacs’s developers. It also means that every function in a package is visible to other packages. If a function is valuable enough, many other packages may come to use it and depend on the details of its behavior.

Oddly, this is not nearly as much of a problem as one might expect. One might hypothesize that Emacs Lisp packages are mostly independent, but of the roughly 1,100 Lisp files included in the standard Emacs distribution, 500 of them have functions used by some other package. My guess is that the maintainers of the packages that are distributed along with Emacs only take care to remain compatible with other such packages, and that those developers form a sufficiently tightly knit group that it’s practical to negotiate incompatible changes as you make them. Packages not included with Emacs probably just break—creating an incentive for developers to get their packages included and join the cabal.

Creeping Featurism

Emacs’s creeping featurism is a direct consequence of its architecture. Here’s the life cycle of a typical feature:

1. When you first notice a feature that would be nice to have, it’s easy to try implementing it; Emacs’s lack of bureaucracy makes the barrier to entry extremely low. Emacs provides a pleasant, interactive environment for Lisp development. The simple buffer Model and automatic display update let you focus on the task at hand.
2. Once you have a command definition that works, you can put it in your *.emacs* file to make it available permanently. If you use it frequently, you can include code there to bind it to a key.
3. Eventually, what began life as a single command may grow into a suite of commands that work together, at which point you can gather them up in a package to share with your friends.
4. Finally, as the most popular packages come to be included in the stock Emacs distribution, its standard feature set expands.

Similar processes are at work within the established codebase. Experience writing your own commands makes the code of Emacs itself that much more comprehensible, so when you notice a potential improvement to an existing command, you might bring up the command's source code (linked to from its help text, as mentioned earlier) and try to implement your improvement. You can redefine Emacs Lisp functions on the fly, making it easy to experiment. If your idea works out, you can put your redefinition in your `.emacs` file for your own use, and post a patch for inclusion in the official sources for everyone else.

Of course, truly original ideas are rare. For long-time users it's a common experience to think of an improvement to Emacs, look through the documentation, and find that someone else has already implemented it. Since the Lisp-friendly cohort of Emacs's user base has been adjusting and adapting Emacs for almost 20 years now, it's usually a given that many people have already been wherever you are now, and someone may have done something about it.

But whether Emacs grows by acquiring new packages or by incorporating patches its users contribute, the growth is a grass-roots process, reflecting its users' interests: the features exist, from the obvious to the strange, simply because someone wrote them and others found them useful. The role of Emacs's maintainers, beyond fixing bugs, accepting patches, and adding useful new primitives, is essentially to select the most popular, well-developed packages that the community is already using for inclusion in the official sources.

If this were the whole story, creeping featurism wouldn't be much of a problem. However, it usually has two unpleasant side effects: the program's user interface becomes too complex to understand, and the program itself becomes difficult to maintain. Emacs manages to ameliorate the former with mixed success, but it escapes the latter rather effectively.

Creeping Featurism and User Interface Complexity

There are two dimensions along which one can assess the complexity of an application's user interface: the complexity of the Model being manipulated, and the complexity of the command set that operates on that Model.

How complex is the Model?

How much does the user need to learn before he can be confident that he's put the application's Model in the state he wants? Is there hidden or obscure state that affects the Model's meaning?

Microsoft Word documents have a complex model. For example, Word has the ability to automatically number the sections and subsections of a document, keep the numbering current as pieces come and go, and update references to particular sections in the text. However, making this feature work as expected requires a solid understanding of Word style sheets. It is easy to make a mistake that has no visible effect on the document's contents, but that prevents renumbering from working properly. (For another example, ask a help desk staffer about "automatically updating styles" in the 2003 edition of Word.)

Emacs avoids this kind of problem by taking the easy way out: it doesn't support style sheets, automatically numbered sections, headers or footers, tables, or any number of other features expected from modern word processors. It's purely a text editor, and an Emacs buffer is just a string of characters. In almost every circumstance, all the buffer state that matters is readily apparent. As a result, one is rarely confused over what Emacs has done to the contents of one's files.

How complex is the command set?

How easy is it to discover the actions that are relevant and useful at any given point? How easy is it to discover features one hasn't used yet? In this sense, Emacs's user interface is quite complex. A freshly started Emacs session without customization recognizes around 2,400 commands and 700 key bindings, and will usually load many more as the session goes on.

Fortunately, a new user need not confront this horde all at once, any more than a Unix user needs to learn every shell command. It's perfectly possible for a new user to treat Emacs like any other editor with a graphical user interface, selecting text with the mouse, moving the cursor with the arrow keys, and loading and saving files with menu commands. Commands left unused don't impinge on the visibility of essential functionality.

However, using Emacs in this fashion isn't much better than using any other editor. Becoming a proficient Emacs user entails reading the manual and the online documentation, and learning to search through these resources effectively. Features such as *grep* and compilation buffers, interactive debugging, and source code indexing are what distinguish Emacs from its peers, but they don't reveal themselves unless you explicitly request them—which you wouldn't do unless you already knew they existed.

To make this kind of exploration a little easier, Emacs also includes the *apropos* family of commands, which prompt for a string or regular expression, and then list commands and customization variables with matching names or documentation strings. Although they're no substitute for reading the manual, the *apropos* commands are effective when you have a general idea what you're looking for.

In terms of this kind of complexity, the Emacs user interface has many of the characteristics common to command-line interfaces: it's possible to have many commands available, the user need not know all of them (or even many of them), and it takes deliberate effort to discover new functionality.

Fortunately, the Emacs community has been effective at establishing conventions that commands should follow, so there's a good deal of consistency from one package to the next. For example, almost all Emacs commands are modeless: the standard commands for moving, searching, switching buffers, rearranging windows, and so on are always available, so you needn't worry about how to "exit" a command. As another example, most commands use standard Emacs facilities to prompt the user for parameters, making prompting behavior consistent from one package to the next.

Creeping Featurism and Maintainability

Obviously, the more code you have, the more effort it takes to maintain it. When a developer's Lisp package is selected for inclusion in the standard Emacs distribution, the lead maintainers invite that package's author to continue maintaining it, so as the number of packages expands, the number of maintainers expands to match. If someone relinquishes responsibility for a package (perhaps because she's too busy or no longer interested), then the lead maintainers must either find a new volunteer or remove the package.

The key to this solution is that Emacs is a collection of packages, not a unified whole. In a sense, the dynamics of Emacs maintenance more closely resemble those of a platform (such as an operating system) than a single application. Instead of a single design team choosing priorities and allocating effort, there is a community of self-directed developers, each pursuing his own ends, and then a process of selection and consolidation that brings their efforts into a single release. In the end, no single person bears the weight of maintaining the entire system.

In this process, the Lisp language acts as an important abstraction boundary. Like most popular interpreted languages, Emacs Lisp largely insulates code from the details of the Lisp interpreter and the underlying processor architecture. Likewise, the editing primitives available to Lisp conceal the implementations of buffers, text properties, and other editing objects; the characteristics visible to Lisp are, for the most part, restricted to those the developers are willing to commit to supporting in the long term. This gives Emacs's core of C code a good deal of freedom to improve and expand, without the risk of breaking compatibility with the existing body of Lisp packages. For example, Emacs buffers have acquired support for text properties, overlays, and multiple character sets while remaining largely compatible with code written before those features existed.

Two Other Architectures

Many applications allow user extensions. Extension interfaces appear in everything from collaborative software development website systems (such as Trac plugins) to word processing software (Open Office's Universal Network Objects) to version control software (Mercurial's extensions). Here I compare Emacs with two architectures that support user extensions.

Eclipse

Although most people know Eclipse as a popular open source integrated development environment for Java and C++, Eclipse proper includes almost no functionality. Rather, it is a framework for plug-ins, allowing components that support specific aspects of development—writing Java code, debugging a running program, or using version control software—to communicate easily with each other. Eclipse's architecture allows programmers with a solid solution to one part of the problem to join their work with others' in order to produce a unified and full-featured development environment.

As a development environment, Eclipse provides valuable features that Emacs lacks. For example, the Java Development Tools plug-ins provide extensive support for refactoring and code analysis. In comparison, Emacs has only a limited understanding of the semantic structure of the programs it edits, and can't offer comparable support.

Eclipse's architecture is nothing if not open-ended, as plug-ins provide nearly all significant functionality. There are no second-class citizens: the popular plug-ins are built on the same interfaces available to others. And because Eclipse gives each plug-in relatively low-level control over its input and display, the plug-in is free to choose whatever Model, View, and Controller best suits its purpose.

However, this approach has a number of drawbacks:

- Eclipse plug-in development is not safe, in the sense attributed to Emacs Lisp code. A buggy plug-in can easily cause Eclipse to crash or lock up. In Emacs, the Lisp interpreter ensures that the user can interrupt runaway Lisp code, and the strong boundary between Lisp code and the Model implementation protects the user's data from the more insidious forms of corruption.
- Because the interfaces that Eclipse plug-ins offer each other are relatively complex, writing an Eclipse plug-in is more like adding a module to a sophisticated application than writing a script. Certainly, these interfaces are what have made Eclipse's features possible, but plug-in authors must pay that price for both ambitious and simple-minded projects.
- A plug-in requires enough boilerplate code that Eclipse includes a plug-in to help people write plug-ins. The Eclipse Plug-in Development Environment can generate code skeletons, write default XML configuration files and manifest files, and create and tear down test environments. Providing a "wizard" to generate boilerplate code automatically can help get a plug-in developer started, but it doesn't reduce the complexity of the underlying interfaces.

The overall effect is to leave Eclipse plug-ins as a rather coarse-grained extension facility. Plug-ins are not suited for quick-and-dirty automation tasks, and they're not especially friendly to casual user-interface experimentation.

This suggests the second question of the three I promised at the beginning of the chapter, one we can ask of any plug-in facility: *what sort of interfaces are available for plug-ins to use?* Are they simple enough to allow the kind of rapid development associated with scripting languages? Can a plug-in developer work at a high level of abstraction, close to the problem domain? And how is the application's data protected from buggy plug-in code?

Firefox

The current generation of sophisticated web applications (Google Mail, Facebook, and so on) makes heavy use of techniques such as *dynamic HTML* and *AJAX* to provide a smooth user experience. These applications' web pages contain JavaScript code that responds to the user's

input locally, and then communicates with the underlying servers as needed. The JavaScript code uses a standard interface, the *Document Object Model*, to navigate and modify the web page's contents, and further standards dictate the page's visual appearance. All modern web browsers implement these standards to some degree.

Although a web browser is not a text editor, there are some striking resemblances between Emacs's architecture and that of a browser:

- Although Emacs Lisp and JavaScript don't resemble each other much at the syntactic level, their semantics have many essential traits in common: like Emacs Lisp, JavaScript is interpreted, highly dynamic, and safe. Both are garbage-collected languages.
- As with Emacs Lisp, it's very practical to begin with a small fragment of JavaScript on a page to improve some minor aspect of its behavior, and then grow that incrementally into something more sophisticated. The barrier to entry is low, but the language scales up to larger problems.
- As in Emacs, display management is automatic. JavaScript code simply edits the tree of nodes representing the web page, and the browser takes care of bringing the display up to date as needed.
- As in Emacs, the process of dispatching input events to JavaScript code is managed by the browser. Firefox takes care of deciding which element of the web page an event was directed at, finds an appropriate handler, and invokes it.

However, Firefox takes the ideas behind these modern web applications a bit further: Firefox's own user interface is implemented using the same underlying code that displays web pages and handles their interactions. A set of packages known as *chrome* describe the interface's structure and style, and include JavaScript code to bring it to life.[†] This architecture allows third-party developers to write *add-ons* that extend Firefox's user interface with new chrome packages. Taking the same techniques even further, developers can replace the standard Firefox chrome altogether and radically reshape the entire user interface—to adapt it for use on mobile devices, for example.

Like Eclipse plug-ins, Firefox chrome packages include a significant amount of metadata. And, resembling Eclipse's Plug-in Development Environment plug-in, there is a Firefox extension to help people write Firefox extensions. So there is still a significant amount of work required up front before one can extend or fix Firefox. However, Firefox's automatic display management and simplified event handling mean that the effort required is still not as high as that needed to write an Eclipse plug-in.

[†] Naturally, JavaScript code used in chrome can read and write preference files, bookmark tables, and ordinary user files—privileges that would never be granted to code downloaded from a web page.

Firefox’s developers are working to improve the performance of its JavaScript implementation. Although this obviously helps users visiting JavaScript-heavy websites, it also allows the Firefox developers to migrate more and more of the browser itself from C++ to JavaScript, a much more comfortable and flexible language for the problem. In this sense, Firefox’s architecture is evolving to look more like that of Emacs, with its all-Lisp Controller.

This suggests the third and final question, one we can ask of any extension language we encounter: *is the extension language the preferred way to implement most new features for the application?* If not, what restrictions discourage its use? Is the language itself weak? Is its interface to the Model cumbersome? Whatever the case, these same weaknesses probably affect extension developers in similar ways, leaving extensions as second-class citizens. (One could ask the analogous question of plug-in interfaces as well.) Like Emacs, Firefox places its extension language at the heart of its architecture, a strong argument that the language’s relationship with the application has been designed properly.

As an avid Emacs user, but one concerned about its future, I’m especially interested in Firefox because it seems so close to Emacs in many ways: a View that provides automatic display management, a Controller based on an interpreted, dynamic language, and a Model that does everything Emacs’s does, and much more. If one were willing to leave behind the accumulated corpus of Emacs Lisp code, a few days’ worth of chrome programming could produce a text editor with an architecture very similar to Emacs’s, but with a much richer model and a much stronger connection to the current frontiers of technology. The most valuable lessons Emacs’s architecture has to teach need not be forgotten.